

Тестовое задание QA (SDET)

УРОВЕНЬ	ВРЕМЯ	ФОРМАТ
Middle / Middle+	3-4 часа	API + SQL

В задании важны не только позитивные сценарии. Нас интересует, как кандидат проверяет идемпотентность, поведение системы при неопределённом результате и расхождения в финансовых данных. Стек для тестов можно выбрать самостоятельно.

Задачи 1 и 2 обязательны. Задача 3 выполняется по желанию и не входит в ориентир 3-4 часа.

1. Автотесты платёжного API

В комплекте находится готовый сервис `payment_sandbox.py`. Это система под тест. Требуется Python 3.9 или новее; внешних зависимостей и дополнительных сервисов нет.

Запуск

```
python3 payment_sandbox.py
# http://127.0.0.1:8080
```

Проверка готовности

```
curl http://127.0.0.1:8080/health
# {"status": "ok"}
```

Подготовка окружения не является частью оценки и должна занять не более 10-15 минут. Docker, Makefile и отдельная инфраструктура не требуются. Если сервис не запускается по инструкции, опишите проблему и свяжитесь с нами. Время на её устранение не входит в лимит задания.

Работайте с сервисом как с чёрным ящиком. Не изменяйте `payment_sandbox.py` и не импортируйте его внутренние функции или состояние в тесты.

Контракт

POST `/v1/payments`

- Заголовок `Idempotency-Key` обязателен.
- Тело запроса: `{ "corridor": "RUB/THB", "amount": 100000, "currency": "RUB", "beneficiary_ref": "..." }`
- 201 - создан новый платёж.
- 200 - тот же ключ и то же тело: возвращается прежний `payment_id`, новый платёж не создаётся.
- 409 - тот же ключ, но другое тело: ошибка, ничего не создаётся.
- 422 - невалидное тело, в том числе некорректная сумма.
- 400 - отсутствует заголовок идемпотентности.

Дополнительные endpoints

- GET /v1/payments/{id} - текущее состояние платежа.
- GET /health - проверка готовности сервиса: 200 {"status": "ok"}.
- GET /debug/payments?beneficiary_ref=<ref> - диагностический endpoint для проверки количества созданных платежей. Возвращает count и массив payments. Параметр beneficiary_ref обязателен; без него сервис отвечает 400.

Симуляция сбоя

Заголовок X-Simulate: fail-after-create заставляет сервер обработать запрос и вернуть 500. Это моделирует ситуацию, когда клиент не знает фактический результат операции: например, из-за таймаута или обрыва соединения.

Минимальное покрытие

- Повтор с тем же ключом и телом не создаёт второй платёж.
- Тот же ключ с другим телом.
- Разные ключи создают разные платежи.
- Сбой на первом запросе и повтор с тем же ключом. Второго платежа быть не должно; побочный эффект следует проверить по уникальному beneficiary_ref.
- Валидация: отсутствующий ключ, некорректная сумма и неподдерживаемая валюта.

Проверяйте HTTP-ответы и фактический результат операции. Если поведение расходится с контрактом, зафиксируйте это в баг-репорте: ожидаемый результат, фактический результат и критичность.

Способ запуска тестов и вспомогательная обвязка сами по себе не оцениваются. Оцениваются корректность проверок, покрытие контракта, контроль побочных эффектов, качество диагностики и ясность баг-репорта.

2. Реконсиляция

В комплект входят файлы recon_our_records.csv и recon_partner_statement.csv. Записи сопоставляются по полю ref.

Загрузите данные в любой SQL-движок, например SQLite или PostgreSQL, и запросами найдите все расхождения. Приложите SQL и коротко опишите, что обнаружено и какие действия потребуются.

Нужно найти как минимум

- платежи, которые есть у нас, но отсутствуют у партнёра;
- платежи, которые есть у партнёра, но отсутствуют у нас;
- несовпадения сумм;
- несовпадения статусов;
- дубли в выписке партнёра.

Совпадающие записи отмечать не требуется.

3. E2E-сценарий (опционально)

Подготовьте один сценарий на Playwright для формы оплаты: ввод данных, переход на страницу подтверждения и проверка финального статуса. Можно использовать простую тестовую страницу или публичный демонстрационный checkout. Важны сам сценарий и стабильность проверок.

Что прислать

- ссылку на репозиторий или архив с тестами по задаче 1;
- SQL-запросы и результаты анализа по задаче 2;
- баг-репорт по найденным расхождениям;
- короткий README: как запустить решение, какие случаи покрыты и что стоило бы добавить при большем времени.

Честно обозначенные ограничения и риски важнее формально идеального, но неполного решения.